

SECE843

Security By Design

Sécuriser une application web Python / Flask
de la conception au déploiement

Master · 2 séances de 3h30 · Application support : **VulnPyApp**

Sommaire

1. **Fondamentaux** — CIA, vocabulaire, OWASP Top 10, RGPD
2. **Séance 1 — Injections** — SQLi, XSS, protections navigateur
3. **Séance 2 — Autorisations & Auth** — CSRF, IDOR, Mass Assignment, Auth, Sessions, Rate Limiting
4. **SDLC sécurisé** — Bandit, Safety, checklist
5. **OWASP API Security Top 10**
6. **VulnPyApp** — les 15 vulnérabilités du support
7. **Exercices & calendrier des rendus**

Partie 1 — Fondamentaux

La triade CIA

Pilier	Objectif	Mécanismes
Confidentialité	L'info n'est lisible que par les autorisés	RBAC, MFA, TLS, chiffrement au repos, HSM/KMS
Intégrité	L'info n'est pas altérée sans autorisation	Hash SHA-256, HMAC, signatures, contraintes BDD, audit log
Disponibilité	Le service reste accessible aux légitimes	Redondance, load balancing, WAF, rate limiting, PRA/PCA

Les **trois piliers sont interdépendants** : un système confidentiel mais indisponible n'est pas sécurisé.

Vocabulaire essentiel

- **Surface d'attaque** : ensemble des points d'entrée exposés (formulaires, APIs, dépendances, infra)
- **Vecteur d'attaque** : chemin emprunté (injection, XSS, élévation, ingénierie sociale)
- **Vulnérabilité → Exploit → Payload**
 - **Vulnérabilité** = la faiblesse
 - **Exploit** = la technique qui en tire parti
 - **Payload** = le code malveillant exécuté

Chaîne d'attaque : découverte → exploit → livraison du payload → impact (exfiltration, escalade, DoS).

Principes Security by Design

1. **Defense in depth** — plusieurs couches de sécurité
2. **Least privilege** — accorder uniquement les droits nécessaires
3. **Fail securely** — en cas d'erreur, l'état reste sûr
4. **Zero trust** — ne jamais faire confiance, toujours vérifier
5. **Security by default** — configuration sécurisée d'origine
6. **Keep it simple** — la complexité est ennemie de la sécurité

CVE & CVSS

- **CVE** (Common Vulnerabilities and Exposures) : identifiant unique d'une faille
- **CVSS** (Common Vulnerability Scoring System) : score de gravité **0** → **10**

Score	Niveau
0.1 – 3.9	Faible
4.0 – 6.9	Moyen
7.0 – 8.9	Élevé
9.0 – 10.0	Critique

Cycle d'une vulnérabilité : *Découverte* → *Divulgation responsable* → *CVE* → *Patch* → *Déploiement*.

OWASP Top 10 — 2021

Code	Catégorie	Couvert en
A01	Broken Access Control	Séance 2 (IDOR)
A02	Cryptographic Failures	Séance 2 (Auth)
A03	Injection (SQL, OS, SSTI)	Séance 1
A04	Insecure Design	Séance 2 (SDLC)
A05	Security Misconfiguration	Séance 1 (CORS/CSP)
A06	Vulnerable & Outdated Components	Séance 2 (Safety/SCA)
A07	Identification & Auth Failures	Séance 2 (Auth/MFA)
A08	Software & Data Integrity Failures	Séance 2 (CSRF)
A09	Logging & Monitoring Failures	Séance 2
A10	Server-Side Request Forgery	Checklist

RGPD pour développeurs (l'essentiel)

Art. 5 — Principes : licéité, finalités limitées, **minimisation**, exactitude, conservation limitée, intégrité.

Art. 25 — Privacy by Design & by Default

- Minimisation (ex. âge plutôt que date de naissance complète)
- Pseudonymisation dans les logs
- MFA proposé dès l'inscription
- Consentement tracé, révocable

En cas de fuite : notification CNIL sous **72 h**, notification aux personnes si risque élevé, registre des violations. Sanctions jusqu'à **4 % CA mondial ou 20 M€**.

Données sensibles → mesures techniques

Type	Risques	Mesures
Mots de passe	Vol, bruteforce	Argon2/bcrypt + sel , HaveIBeenPwned
Email/Nom	Phishing, profilage	Chiffrement au repos, pseudonymisation logs
CB / IBAN	Fraude, non-conformité PCI-DSS	Tokenisation, chiffrement colonne
Cookies de session	Hijacking, CSRF	<code>HttpOnly + Secure + SameSite=Strict</code>
Logs	Fuite de PII	Filtrage des données sensibles, rétention
Fichiers uploadés	Path traversal, RCE	MIME check, renommage, hors webroot
Clés / secrets	Compromission totale	Variables d'env, HSM/KMS, jamais en Git

Séance 1 — Injections

SQLi · XSS · SOP / CORS / CSP

Application support — VulnPyApp

- **Flask** monofichier, volontairement vulnérable (v1.0)
- 15 vulnérabilités numérotées, marquées  **VULN #N** dans le code
- **Jumelle** sécurisée `vulnpyapp_remediated/` (v2.0), correctifs marqués  **FIX #N**
- Comptes de test :
 - `admin@vulnpyapp.local` / Admin123!
 - `alice@vulnpyapp.local` / Alice123!
 - `bob@vulnpyapp.local` / Bobby123!

```
cd vulnpyapp && pip install -r requirements.txt  
python init_db.py && python app.py # http://localhost:5000
```

 **Jamais en production** — usage exclusivement local (Docker, VM, localhost).

Injection SQL — principe

```
# 🚩 VULNÉRABLE — concaténation directe
query = f"SELECT * FROM users WHERE email='{email}' AND password='{password}'"
cursor.execute(query)
```

Avec `email = "admin@vulnpyapp.local' --"` :

```
SELECT * FROM users WHERE email='admin@vulnpyapp.local' --' AND password='...'
-- le `--` neutralise la suite : bypass du mot de passe
```

L'entrée utilisateur devient **partie de la commande SQL** au lieu d'être une simple donnée.

Injection SQL — types

- **In-band**
 - *Error-based* : exploiter les messages d'erreur
 - *UNION-based* : récupérer les données via `UNION SELECT`
- **Blind**
 - *Boolean-based* : déduire via vrai/faux
 - *Time-based* : déduire via délais (`randomblob` , `SLEEP`)
- **Out-of-band** : exfiltration via DNS / HTTP externe

Outil de référence : **sqlmap** (`sqlmap -u "http://.../search?q=test" --dbs`).

SQLi — correction : requêtes paramétrées

```
# ✅ sqlite3 avec placeholders
cursor.execute("SELECT * FROM users WHERE email=? AND password=?", (email, password))
```

```
# ✅ ORM SQLAlchemy (recommandé)
user = User.query.filter_by(email=email).first()
if user and user.check_password(password):
    login_user(user)
```

Défense en profondeur : validation Pydantic / Marshmallow, moindre privilège sur le compte DB, logs.

XSS — trois variantes

Type	Vecteur	Persistance	Cible
Reflected	Payload dans l'URL, reflété dans la réponse	Non	Victime du lien
Stored	Payload stocké en BDD (commentaire, profil...)	Oui	Tous les visiteurs
DOM-based	JS client manipule <code>innerHTML</code> à partir d' <code>URL</code> / <code>hash</code>	Non	Navigateur local

Impacts : vol de cookie, défiguration, keylogger, actions API non autorisées.

XSS — corrections en Flask

```
# ✅ Jinja2 auto-escape par défaut  
return render_template('search.html', query=query)
```

```
# ✅ Échappement manuel  
from markupsafe import escape  
return f"<h1>Résultats pour : {escape(query)}</h1>"
```

```
# ✅ HTML riche → bleach avec allowlist  
import bleach  
clean = bleach.clean(raw, tags=['b', 'i', 'em', 'strong', 'p', 'br'], strip=True)
```

⚠️ **Ne jamais utiliser** `{{ var | safe }}` sur des données utilisateur.

Protections navigateur

SOP (Same-Origin Policy) : Origine = *Protocole* + *Domaine* + *Port*. Isolation par défaut.

CORS : exceptions contrôlées à SOP via headers serveur. ❌ Jamais `origins="*" + credentials=True`.

CSP (Content Security Policy) : whitelist des sources de scripts, styles, images, frames.

```
csp = {  
    'default-src': "'self'",  
    'script-src': ["'self'", "'nonce-{nonce}'"],  
    'frame-ancestors': "'none'",  
    'object-src': "'none'"  
}  
Talisman(app, content_security_policy=csp, content_security_policy_nonce_in=['script-src'])
```

HSTS : `Strict-Transport-Security` force HTTPS.

Séance 2 — Autorisations & Authentication

CSRF · IDOR · Mass Assignment · Auth · Sessions · Rate Limiting

CSRF — principe

L'attaquant fait exécuter à un navigateur **déjà authentifié** une requête sur le site cible.
Le navigateur joint **automatiquement** le cookie de session, sans que l'attaquant ait à le voler.

Scénario : Alice est connectée à VulnPyApp. Elle visite `blog-piege.com` qui auto-soumet en JS un `<form action="http://localhost:5000/profile/update" method="POST">` .

→ Le profil d'Alice est modifié à son insu.

CSRF — protections

```
# ✅ Méthode 1 : tokens CSRF avec Flask-WTF
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect(app)
# Dans le template : {{ form.csrf_token }}
```

```
# ✅ Méthode 2 : cookies SameSite (défense en profondeur)
app.config.update(
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE='Strict',
)
```

Méthode 3 : vérification des en-têtes `Origin` / `Referer` côté serveur.

Authentication vs Autorisation

Concept	Question	Mécanismes
Authentication (AuthN)	<i>Qui êtes-vous ?</i>	Login, MFA, certificat
Autorisation (AuthZ)	<i>Que pouvez-vous faire ?</i>	RBAC, ABAC, ACL

Modèles d'accès :

- **RBAC** — rôles prédéfinis (`admin` , `user`)
- **ABAC** — attributs contextuels (service, heure, statut)
- **ACL** — liste explicite par ressource

⚠ La vérification d'autorisation se fait **toujours côté serveur**.

IDOR — Insecure Direct Object Reference

```
# 🚩 VULNÉRABLE
@app.route('/api/orders/<int:order_id>')
def get_order(order_id):
    return jsonify(Order.query.get(order_id).to_dict())
```

```
# ✅ Filtrage à la source
order = Order.query.filter_by(
    id=order_id, user_id=current_user.id
).first_or_404()
```

Autres parades : décorateur `@require_ownership` , **UUID publics** au lieu d'IDs séquentiels.

Mass Assignment

```
# 🚩 VULNÉRABLE
data = request.form.to_dict()
user = User(**data)          # is_admin=true passe sans contrôle !
```

```
# ✅ Schéma Pydantic / Marshmallow avec allowlist
class UserUpdateSchema(BaseModel):
    email: EmailStr | None = None
    name: constr(max_length=100) | None = None
class Config:
    extra = 'forbid'          # rejette les champs inconnus
```

Même logique pour les API : ne **jamais** faire `setattr(user, k, v)` en boucle sur tout le JSON.

SSTI & ReDoS — pièges fréquents

SSTI (Server-Side Template Injection) — peut mener à un RCE complet :

```
# 🚩 VULNÉRABLE
return render_template_string(f"<h1>Hello {name}</h1>")
# ✅ Template statique, name passé en variable
return render_template_string("<h1>Hello {{ name }}</h1>", name=name)
```

ReDoS — regex à backtracking exponentiel.

- Limiter la taille d'entrée
- Préférer `re2` (linéaire) ou des validateurs dédiés (`email_validator`)

Stockage des mots de passe

Méthode	Verdict
Stockage en clair	🔥 Catastrophe
MD5 / SHA-256 sans sel	💣 Cassé / bruteforce GPU
SHA-256 + sel non itéré	⚠️ Insuffisant
bcrypt (rounds ≥ 12)	✅ Bon
Argon2id (OWASP)	✅ Excellent

```
from argon2 import PasswordHasher
ph = PasswordHasher(time_cost=3, memory_cost=65536, parallelism=4)
hashed = ph.hash(password)
```

Politique : ≥ 12 caractères, complexité, check HaveIBeenPwned, **timing constant** pour éviter l'énumération.

MFA & réinitialisation

Multi-facteurs — combiner ≥ 2 parmi :

- *Ce que je sais* — mot de passe, PIN
- *Ce que je possède* — TOTP, FIDO2/WebAuthn
- *Ce que je suis* — biométrie

Reset de mot de passe sécurisé :

- Token cryptographique (`secrets.token_urlsafe(32)`)
- **Hash du token stocké** (pas le token en clair)
- Expiration courte (1 h) + usage unique
- Réponse identique que le compte existe ou non
- Invalidation des autres sessions après reset

Sessions & cookies

```
SESSION_COOKIE_NAME = 'app_session'    # pas le nom par défaut
SESSION_COOKIE_SECURE = True            # HTTPS only
SESSION_COOKIE_HTTPONLY = True          # inaccessible via JS
SESSION_COOKIE_SAMESITE = 'Strict'      # anti-CSRF
PERMANENT_SESSION_LIFETIME = timedelta(hours=1)
```

À chaque connexion : **régénérer la session** (anti-fixation) — `session.clear()` puis nouvelles données (`session_id` , `ip` , `user_agent` , `created_at`). Validation à chaque requête.

Headers de sécurité (Talisman)

```
Talisman(app,  
    force_https=True,  
    strict_transport_security=True,  
    strict_transport_security_max_age=31536000,  
    content_security_policy=csp,  
    frame_options='DENY',  
    referrer_policy='strict-origin-when-cross-origin',  
    session_cookie_secure=True,  
    session_cookie_http_only=True)
```

À compléter : `X-Content-Type-Options: nosniff` , `Cross-Origin-Opener-Policy: same-origin` ,
`Cache-Control: no-store` sur les pages sensibles.

Rate limiting

```
from flask_limiter import Limiter
limiter = Limiter(app=app, key_func=get_remote_address,
                  default_limits=["200/day", "50/hour"],
                  storage_uri="redis://localhost:6379")

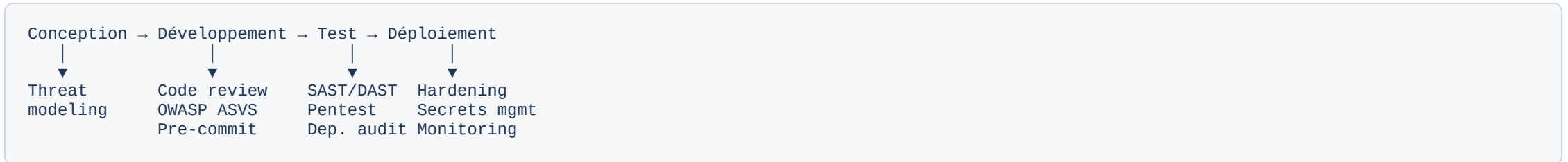
@app.route('/login', methods=['POST'])
@limiter.limit("5 per minute; 20 per hour")
def login(): ...
```

Stratégies : *fixed window, sliding window, token bucket, leaky bucket*.

Réponse 429 Too Many Requests avec en-tête Retry-After .

Compléments : **CAPTCHA v3** (score), détection d'anomalies (pays, device, horaire inhabituel).

SDLC sécurisé



Outils Python :

- **SAST** : bandit , semgrep
- **SCA** : safety , pip-audit
- **Secrets** : detect-secrets
- Tous intégrables en **pre-commit** + CI/CD

Checklist Security by Design

- **Auth** : Argon2/bcrypt, MFA, anti-énumération, régénération de session
- **AuthZ** : moindre privilège, vérif serveur, anti-IDOR, anti-mass assignment
- **Données** : chiffrement repos + transit, minimisation, no-PII-in-logs
- **Validation** : Pydantic/Marshmallow, requêtes paramétrées, auto-escape
- **Comm** : HTTPS + HSTS, CSP stricte, cookies `Secure/HttpOnly/SameSite`, CORS restrictif
- **Infra** : WAF, secrets en vault, dépendances scannées, backups chiffrés
- **Monitoring** : SIEM, alertes, IR plan, bug bounty
- **Process** : threat modeling, code review, SAST CI, pentest annuel

OWASP API Security Top 10 (2019)

Code	Catégorie	Module
API1	Broken Object Level Authorization (BOLA)	2.2 IDOR
API2	Broken User Authentication	2.4 Auth
API3	Excessive Data Exposure	2.3 DTO
API4	Lack of Resources & Rate Limiting	2.6
API5	Broken Function Level Authorization	2.2 RBAC
API6	Mass Assignment	2.3
API7	Security Misconfiguration	2.5
API8	Injection	Séance 1
API9	Improper Assets Management	2.7 SDLC
API10	Insufficient Logging & Monitoring	2.5

VulnPyApp — les 15 vulnérabilités

Cible des exercices · Source du miroir « vulnérable / remédié »

Les 15 vulnérabilités du support (1/2)

#	Vulnérabilité	CWE	Route
1	SQL Injection (login)	CWE-89	/login
2	SQL Injection (search)	CWE-89	/search
3	XSS Reflected	CWE-79	/search
4	XSS Stored	CWE-79	/comments
5	XSS DOM-based	CWE-79	/profile
6	CSRF	CWE-352	/profile/update
7	IDOR	CWE-639	/api/orders/<id>
8	Mass Assignment	CWE-915	/register

Les 15 vulnérabilités du support (2/2)

#	Vulnérabilité	CWE	Route
9	SSTI (Jinja2)	CWE-1336	/hello
10	Path Traversal	CWE-22	/download
11	Command Injection	CWE-78	/ping
12	Hashage faible (MD5)	CWE-327	login system
13	Cookies non sécurisés	CWE-614	sessions
14	Absence de rate limiting	CWE-307	/login
15	Headers de sécurité absents	CWE-693	global

Chaque vulnérabilité est doublée d'un correctif  **FIX #N** dans `vulnpyapp_remediated/`, documenté dans `docs/guide_correction.md`.

Exercices

Séance 1 · Séance 2 · Projet final

Exercice 1.A — Cartographie & RGPD (*lab guidé*)

Durée : 45 min en séance · **Rendu** : en fin de séance

Partie 1 — Cartographie automatisée (15 min)

- Script `cartographie.py` qui crawl VulnPyApp
- Inventaire : endpoints, formulaires, APIs, données sensibles
- Rapport Markdown généré

Partie 2 — Analyse RGPD (30 min)

- Matrice des risques (≥ 8 fonctionnalités, score impact $\times$ probabilité)
- Checklist RGPD détaillée (minimisation, consentement, droits, etc.)
- 3 non-conformités majeures avec recommandations

Exercice 1.B — CTF SQLi & XSS (20 %)

Durée : 2 h en séance + 1 semaine · **Binôme** · ZIP sur la plateforme

#	Challenge	Pts
1	SQLi Login Bypass	10
2	SQLi UNION (dump users + hashes)	15
3	IDOR / Information Disclosure (<code>/api/users/<id></code>)	20
4	XSS Réfléchie	5
5	XSS Stockée (vol cookie + keylogger)	10
—	Corrections (code + tests)	40

Bonus : CSP fonctionnelle (+5), correction IDOR complète (+3), bypass filtre XSS (+3).

Exercice 1.C — Quiz fondamentaux (10 %)

Durée : 15 min en fin de séance · QCM 20 questions

Thèmes :

- Triade CIA, concepts fondamentaux
- Principes RGPD applicables au développeur
- Mécanismes d'injection SQL
- Types et fonctionnement des XSS
- SOP, CORS, CSP

Exercice 2.A — Revue de Code (15 %)

Durée : 45 min en séance · **Individuel** · Documents autorisés

Code Flask de **gestion de commandes** (`orders_api.py`) à analyser.

Objectif : trouver **6 vulnérabilités** parmi les **9 présentes**.

Pour chacune : *ligne · nom · CWE · CVSS · correction*.

Pièges présents : secret key statique, MD5, SQLi, IDOR, mass assignment, SSTI, path traversal, command injection, debug en prod.

6 trouvées = note max. 8-9 trouvées = bonus +5 pts.

Exercice 2.B — Audit & Tests (25 %)

Durée : 1h30 en séance + 1 semaine · **Binôme** · Dépôt Git + rapport PDF

Partie	Contenu	Points
A — Audit outillé	Rapport Bandit, Safety/pip-audit, checklist OWASP Top 10	40
B — Tests pytest	SQLi (×5), XSS (×5), Auth (×4), Access Control (×4)	40
C — Rapport d'audit	Méthodologie OWASP Testing Guide, recommandations P1/P2	20

Bonus : tests avancés (+10), pipeline CI/CD (+5), rapport PDF pro (+3).

Calendrier des rendus

Exercice	Description	Deadline	Poids
1.A	Lab guidé (en séance)	Séance 1	—
1.B	CTF SQLi & XSS	S1 + 7 j	20 %
1.C	Quiz fondamentaux (QCM)	Fin séance 1	10 %
2.A	Revue de code (en séance)	Séance 2	15 %
2.B	Audit + pytest	S2 + 7 j	25 %
3 (projet)	Sécurisation app Django	S3 + 14 j	30 %
		TOTAL	100 %

Pénalités : retard –10 pts/jour, code copié sans compréhension –30 pts.

Pour aller plus loin

Ressources

- **OWASP Top 10 2021** — <https://owasp.org/Top10/2021/>
- **OWASP API Security Top 10 2019** — <https://owasp.org/www-project-api-security/>
- **OWASP Testing Guide v4.2** — méthodologie d'audit
- **CWE** — <https://cwe.mitre.org/>
- **PortSwigger Web Security Academy** — labs en ligne
- **HaveIBeenPwned API** — vérification de mots de passe compromis
- **Dépôt du cours :**
 - `vulnpyapp/` — app vulnérable
 - `vulnpyapp_remediated/` — app sécurisée jumelle
 - `docs/guide_correction.md` — correctifs pas-à-pas

Merci

SECE843 — Security By Design

Questions ?

Code & supports : `vulnpyapp/`, `vulnpyapp_remediated/`, `docs/`, `seance1.md`, `seance2.md`